Scrape Board

A full-stack Kanban project manager with AI task breakdown (Gemini) and website-to-task conversion (Firecrawl) — built solo end-to-end, including a live deployment to shared hosting.

Problem

Lightweight project trackers require every task to be typed manually, and turning reference material (a spec page, an article, a competitor site) into actionable to-dos is repetitive. This project tests whether generative task breakdown and structured web extraction can be embedded directly into a task manager's core workflow, and whether the result can be shipped to real, ordinary shared hosting rather than a Node-friendly platform.

Approach & Key Decisions

- Schema-first: five tables (boards, columns, tasks, subtasks, ai_breakdown_sessions), RLS enabled on every table so authorization lives in Postgres, not the frontend.
- One integration contract for both AI/automation paths: unstructured input in, external API call, strictly-typed structured output, fail loudly and specifically on bad/missing keys.
- Trade-off accepted: Gemini and Firecrawl are called client-side with keys baked into the public JS bundle at build time. Right call for shipping fast end-to-end; a production version would proxy both through a backend/edge function.
- Deployment trade-off: shipped to existing GoDaddy shared hosting (no Node runtime) instead of a Vercel/Netlify-style platform, by treating Vite's build output as what it actually is — plain static files.

What It Does

- Auth & multi-user Kanban boards (Supabase Auth + RLS)
- Drag-and-drop tasks across To-Do / In Progress / Done, with subtasks and due dates
- AI Task Breakdown: task description → Gemini returns 3–7 structured subtasks
- Website Scraper → Tasks: URL → Firecrawl extracts titles/headings/links → selected items become tasks
- Light/dark theming; deployed and reachable as a live subdomain, not just a local dev build

Getting It Running — Accounts & Deployment

- Supabase: create project, keep “Enable Data API” on / “Auto-expose new tables” off, run the schema migration, then explicitly GRANT table privileges to the authenticated role (RLS alone doesn't grant access — it only filters after access is already permitted).
- Firecrawl (firecrawl.dev) and Google Gemini (aistudio.google.com) free-tier API keys, set as VITE_* env vars — Vite bakes these into the build, so set them before npm run build.
- Deploy: npm run build → upload only dist/'s contents to the host's document root → add an .htaccess rewrite so React Router deep links don't 404 on Apache. Rebuilds wipe dist/, including .htaccess — re-add it each time.

Results & Learnings

- RLS held up under direct attempts to query other users' data from the browser console.
- Biggest reliability risk was LLM output not matching the expected JSON shape — defensive parsing mattered more than prompt engineering.
- Grants and RLS are two separate layers in Supabase; disabling table auto-exposure (the secure default) silently blocks all API access until grants are added.
- Deploying a Vite SPA to shared hosting is mechanically simple once the build-output-is-just-static-files mental model clicks; the real friction was operational — rebuild/re-upload cycles, retired Gemini model aliases, and the deprecated Firecrawl /v0 endpoint.

Tech Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite
Styling	Tailwind CSS, shadcn/ui
Backend / DB	Supabase (PostgreSQL, Auth, Row-Level Security)
Drag & Drop	@dnd-kit
AI	Google Gemini API (gemini-2.5-flash)
Scraping	Firecrawl API (/v1/scrape)
Routing	React Router v7
Hosting	GoDaddy shared hosting (static files + Apache .htaccess)